



Technical Document

Manage Ticket Tracking — SDD

SOFTWARE DESIGN DOCUMENT
Manage Ticket Tracking · Sprint Board

Software Design Document (SDD) — Manage Ticket Tracking

Feature: Manage Ticket Tracking (Sprint board) **Apex controller:** force-app/main/default/classes/controller/manageTicketTracking/ManageTicketTrackingController.cls **LWC:** force-app/main/default/lwc/manageTicketTracking/ **Date:** 2026-06-21

Image references below follow the marker convention used across these design documents: the image file name is wrapped between ##### lines, immediately before and after the embedded screenshot.

1. Architecture Overview

```
- manageTicketTracking (parent LWC) -----
Principal state: _sprint = { ...sprint, tickets[] } (de-normalized)
  each ticket carries: ticketType{ workflowTransitions[] }, ticketTypeName,
  assigneeName, isEndStatus, key + _statuses[] (the board lanes)
UI-only state (outside principal): _activeTicketViewId, drag state,
  _searchTerm/_ticketSearchResults, _linkedTo/_subtasks/_history/_comments
  target ids
Derived getters: columns · activeTicketViewModel · sprintDateRange ·
  storyPointsPercent · isTicketViewOpen · ticketViewSearchOptions

Children
  c-ticket-board-column (one per status): header(name+count) · cards ·
    'Drop here' when isValidTarget · card click -> ticketviewopen ·
    dragstart/drop -> status change
  c-ticket-view (centered modal): summary · status · description ·
    linked work items · sub-tasks · history · comments
```

```
| imperative writes + @wire(cacheable=true) reads
```

```
ManageTicketTrackingController
  → StatusService · WorkflowService · ProjectService · TicketService · SprintService
  → Status_c · WorkflowTransition_c · Ticket_c · Sprint_c · Subtask_c · ...
```

Response envelope. Every controller method returns `ApiResponse { success, message, data }`. The LWC treats `success === false` (or a thrown Apex error) as failure and toasts it via `_showError`; successful writes patch the principal state from the returned data, never from an optimistic draft.

Data shape used by the page (from `loadManageTicketTrackingPage`): { `status`, `workflows`, `sprint_tickets`, `members`, `ticketTypes`, `sprint` }. `buildSprintTickets` nests each ticket's type and that type's workflow transitions onto the ticket, and derives `isEndStatus` from the statuses whose `isEnd__c` is true — so the board needs no further round-trips to compute columns or valid drop targets.

2. API Calls (Apex @AuraEnabled methods)

Legend: **C** = cacheable=true (wire-friendly), **I** = imperative (DML). All methods return `ApiResponse{success,message,data}`.

2.1 Page bootstrap & search

Method	Kind	Params	Returns (`data`)	LWC usage
loadManageTicketTrackingPage	C	projectId	{status, workflows, sprint_tickets, members, ticketTypes, sprint}	_loadData() imperative call on mount
loadTicketBySearchTerm	C	projectId, searchTerm	Ticket__c[] (SOSL, special chars escaped)	@wire — ticket-view link search
loadTicketLinkTypes	C	projectId	{ticketLinkTypes}	@wire — link-type combobox options

2.2 Ticket status change (board move + ticket-view combo)

Method	Kind	Params	Effect
changeTicketState	I	ticketId, fromStatusId, toStatusId	requireTransitionAllowed + requireValidateFieldsPass; on pass writes CurrentState__c; if the new status is an end status bumps the sprint ended SP; returns {isEndStatus, updatedSprint}

2.3 Ticket inline edits & links

Method	Kind	Params	Effect
updateTicketSummary	I	ticketId, summary	inline summary edit; returns Ticket__c
updateTicketDescription	I	ticketId, description	rich-text description edit; returns Ticket__c
loadTicketLinkedTo	C	ticketId	{ticketLinkTo} – the ticket's links
linkToTicket	I	fromTicketId, toTicketId, linkType	create link; returns {ticketLink} DTO
createTicketLink	I	fromTicketId, toTicketId, linkType	create link (no DTO returned)

2.4 Sub-tasks

Method	Kind	Params	Effect
loadSubtasks	C	ticketId	Subtask__c[] for the ticket
createSubtask	I	summary, ticketId, description, assigneeId, currentStateId, storyPoint, startDate	create sub-task (summary required; rest optional); returns Subtask__c

2.5 History & comments

Method	Kind	Params	Effect
loadTicketHistory	C	ticketId	TicketHistoryDto[] (activity feed)
loadTicketComments	C	ticketId	TicketCommentDto[] (newest first)
createTicketComment	I	ticketId, message	add comment; returns TicketCommentDto

2.6 Wires actually bound in manageTicketTracking.js

- @wire(loadTicketLinkTypes, { projectId: '\$_projectId' }) → _ticketLinkTypes
- @wire(loadTicketLinkedTo, { ticketId: '\$_linkedToTargetTicketId' }) → patches the linkedTo slice
- @wire(loadTicketBySearchTerm, { projectId: '\$_projectId', searchTerm: '\$_searchTerm' }) → _ticketSearchResults
- @wire(loadSubtasks, { ticketId: '\$_subtasksTargetTicketId' }) → patches the subtasks slice
- @wire(loadTicketHistory, { ticketId: '\$_historyTargetTicketId' }) → patches the history slice
- @wire(loadTicketComments, { ticketId: '\$_commentsTargetTicketId' }) → patches the comments slice; the result is kept so re-expanding the same ticket re-fetches via refreshApex

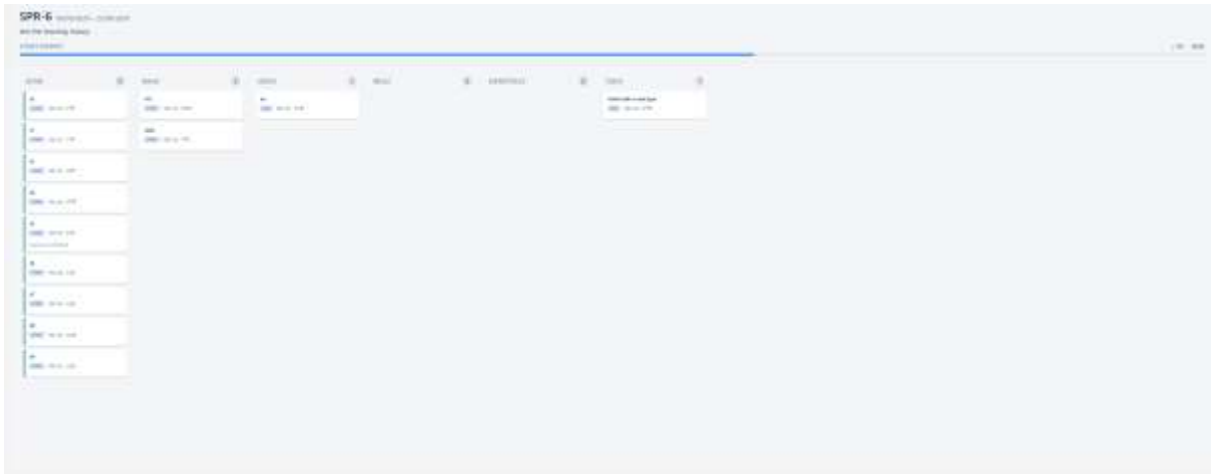
All writes (changeTicketState, updateTicketSummary, updateTicketDescription, linkToTicket, createSubtask, createTicketComment) and the initial page load are imperative Apex calls inside handlers (then/catch/finally), toasting via _showSuccess / _showError.

3. UI / UX Specification

3.1 Page load — sprint header, story-point line, status columns

On load the page shows the active sprint header (name and start → end date range, where the end is derived from start + Duration__c, plus the goal) and the Story Points line (ended / total SP and percent with a progress bar). Beneath it the board renders one column per project status (in order); each column header shows the status name and its ticket count, and holds 0..many ticket cards. If there is no in_progress sprint a “No active sprint” banner replaces the board.

Manage-ticket-tracking-default.png
#####



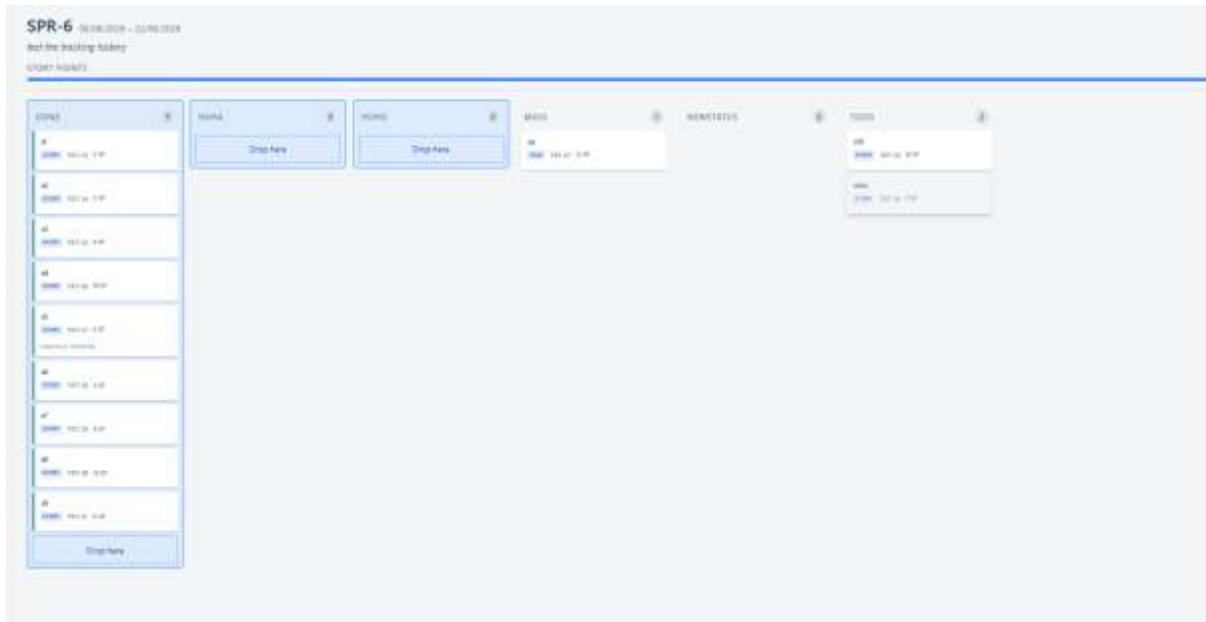
3.2 Ticket card (tracking-card) & the Done (green) border

Each card (`c-ao-ticket-item`, `variant="tracking-card"`) shows the ticket name, ticket type, summary, story points, and the assigned member name. A card whose current status is an end status (`isEndStatus`, from statuses with `isEnd__c = true`) carries `data-end-status="true"`, which the column CSS renders as a 3px green left border (`#00875A`) to signal the ticket has finished. Clicking a card dispatches `ticketviewopen` and opens the ticket-view pop-up.

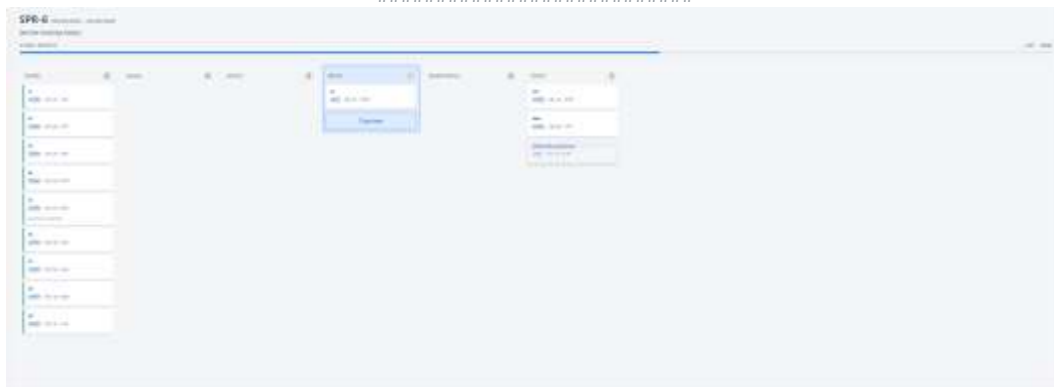
3.3 Drag-and-drop — type-aware drop targets

On drag start the system highlights only the columns reachable from the card's current status for that ticket type (`getValidTargetStatusIds` reads the type's own `workflowTransitions`); valid columns gain the `valid-target` style and a "Drop here" hint, and only they accept a drop. Because each ticket type points at its own workflow, a Story and a Task expose different reachable columns from the same status.

Ticket-type-story-move.png
 #####



Ticket-type-task-move.png
 #####



3.4 Ticket view pop-up (c-ticket-view)

Clicking a card opens the ticket view as a modal centered on the screen over a backdrop (clicking the backdrop or × closes it). It shows the TKT-id, summary, current status combo, description, and the Linked work items / Sub-tasks / History / Comments sections. Status changes here run the same workflow + validate-field gate as a board move.

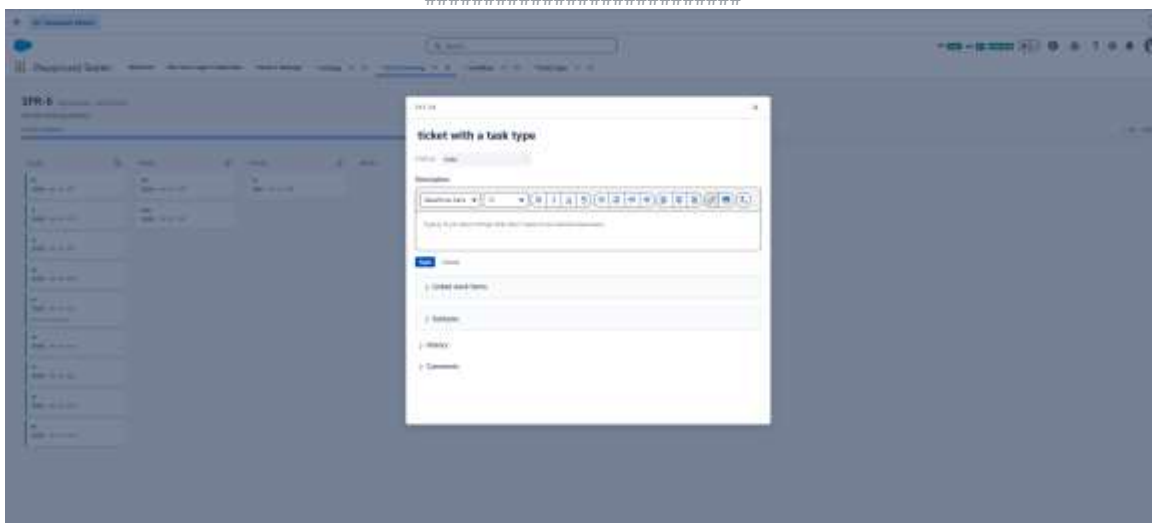
Ticket-view-pop-up.png
 #####



3.5 Edit description (rich text)

Clicking the description opens a rich-text editor with Save / Cancel; Save dispatches ticketdescriptionupdate → updateTicketDescription and patches the principal ticket from the response.

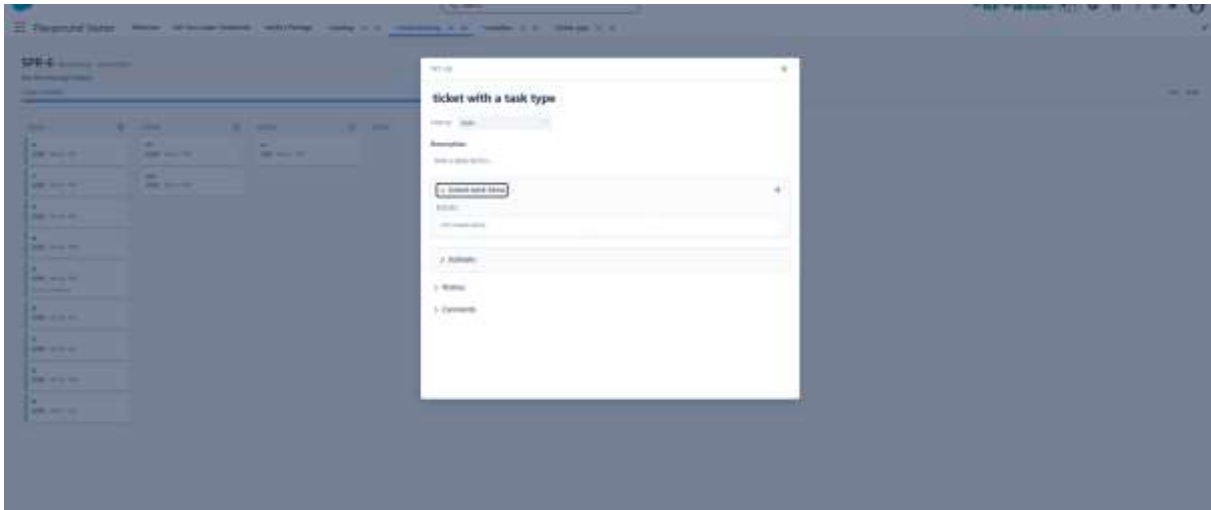
Ticket-view-click-on-description.png
 #####



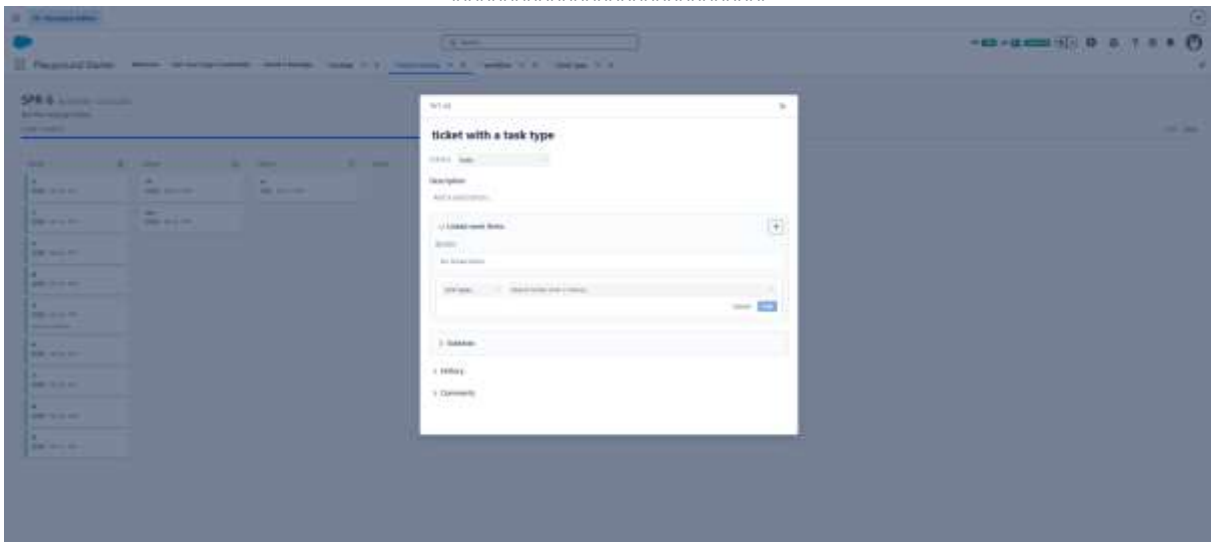
3.6 Linked work items

Expanding Linked work items fires loadTicketLinkedTo and lists the ticket's links; an empty list shows "No linked items." Clicking + reveals the add-link row: choose a Link type and search a target ticket (loadTicketBySearchTerm, min 2 characters), then Link (linkToTicket appends the returned link).

Ticket-view-expand-ticket-view.png
 #####



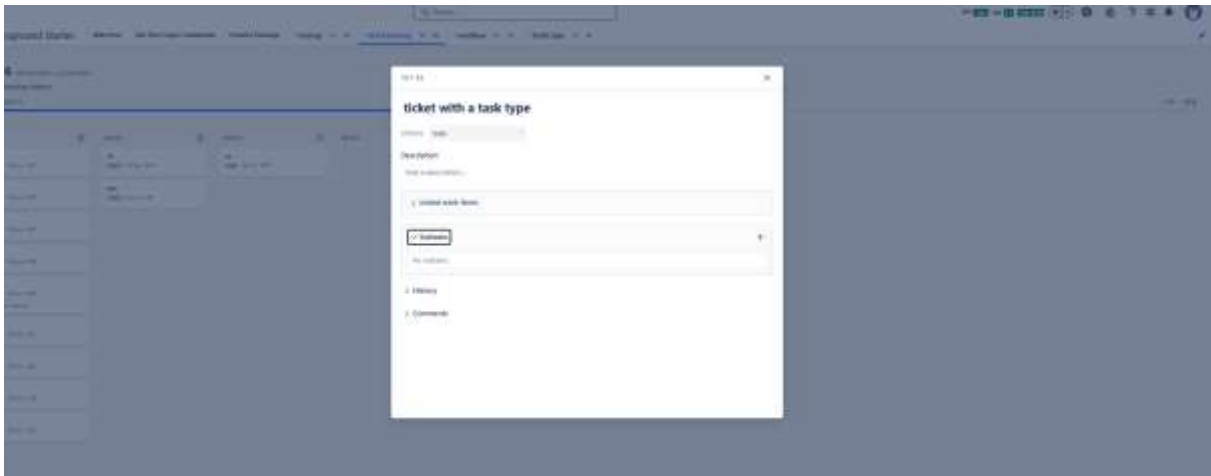
Ticket-view-click-plus.png
 #####



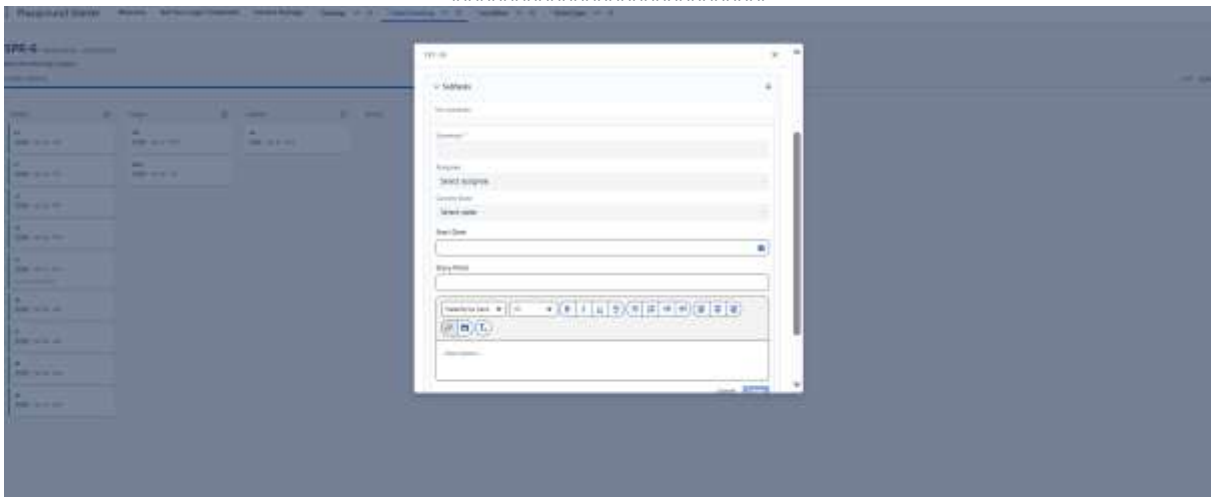
3.7 Sub-tasks

Expanding Sub-tasks fires loadSubtasks and lists them; empty shows “No subtasks.”
 Clicking + opens the create-sub-task form: Summary (required) plus optional Assignee, Current State, Start Date, Story Point, and a rich-text Description; Create calls createSubtask and appends the result.

Ticket-view-expand-subtask.png
 #####



Ticket-view-click-plus-subtask.png
#####



3.8 History

Expanding History fires loadTicketHistory and shows the ticket's activity feed (e.g. "Ticket created"), each entry with its author and timestamp.

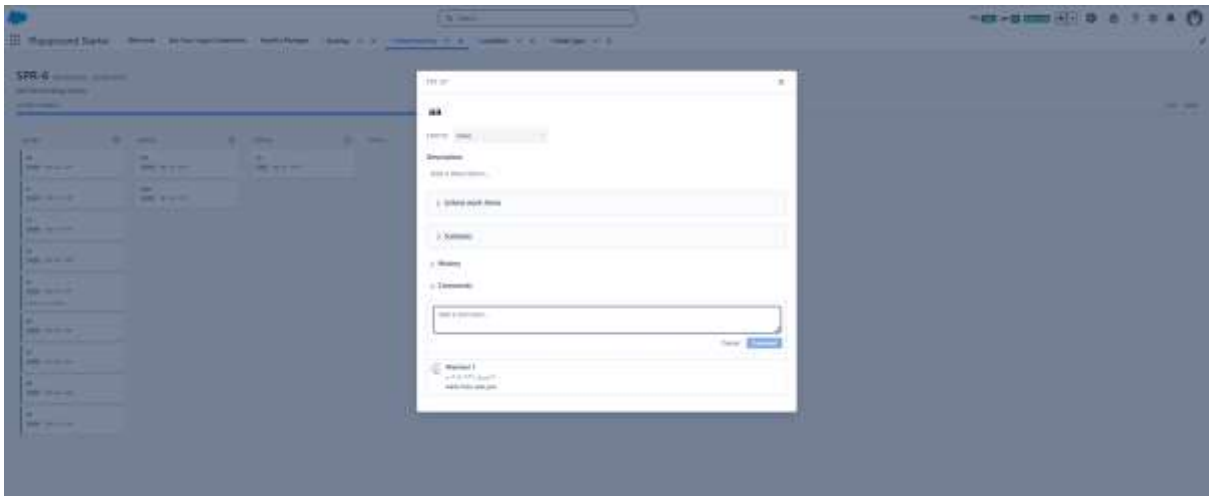
Ticket-view-expand-history.png
#####



3.9 Comments

Expanding Comments fires loadTicketComments and shows the add-comment box and the existing comments (newest first), each with author and timestamp; posting calls createTicketComment and prepends it. Re-expanding the same ticket refreshes via refreshApex.

Ticket-view-expand-comment.png
#####



4. Traceability (SRS → API)

SRS	API call(s) / mechanism
FR-1..FR-3 load & active sprint	loadManageTicketTrackingPage (SprintService.loadActiveSprint)
FR-4/FR-5 sprint header & SP line	loadManageTicketTrackingPage → sprintDateRange, storyPointsPercent
FR-6..FR-8 columns & cards	buildColumns / buildSprintTickets (derived from the page load)
FR-9 Done green border	isEndStatus (statuses.isEnd__c) → data-end-status CSS
FR-10..FR-14 move	getValidTargetStatusIds, findTransitionId (client) → changeTicketState
FR-15..FR-17 open/close/summary	ticketviewopen, closeticketview, updateTicketSummary
FR-18 status from view	changeTicketState
FR-19 description	updateTicketDescription
FR-20/FR-21 links	loadTicketLinkedTo, loadTicketLinkTypes, loadTicketBySearchTerm, linkToTicket
FR-22/FR-23 sub-tasks	loadSubtasks, createSubtask
FR-24 history	loadTicketHistory
FR-25/FR-26 comments	loadTicketComments, createTicketComment